Design and Analysis of Software Systems

Assignment 2 — Integration Testing Report

StreetRace Manager

Roll Number: 2024115006

March 23, 2026

Contents

1 Overview **2**

1.1 Modules 2

1.2 Business Rules Under Test 2

2 Test Suite 1: Registration & Crew Management **3**

3 Test Suite 2: Race, Crew Management & Inventory **5**

4 Test Suite 3: Results, Race & Inventory **6**

5 Test Suite 4: Missions, Crew Management & Inventory **7**

6 Test Suite 5: Leaderboard & Results **8**

7 Test Suite 6: Finance & Inventory **9**

8 Summary of All Errors Found **10**

9 Test Results Summary **11**

1 Overview

This report documents the integration testing performed on the **StreetRace Manager** system. The system is composed of eight modules that together manage crew members, vehicles, races, results, missions, leaderboards, and finances for an underground street racing operation.

Integration testing validates that these modules interact correctly — specifically that data flows accurately between them and that cross-module business rules are enforced.

1.1 Modules

`Module`	**Responsibility**
`registration.py`	Registers crew members, stores name and role
`crew_management.py`	Manages roles and skill levels
`inventory.py`	Tracks cars, parts, tools, and cash balance
`race.py`	Creates races and assigns drivers and cars
`results.py`	Records outcomes, updates rankings, handles prize money
`missions.py`	Assigns missions and verifies required roles
`leaderboard.py`	Displays driver standings and statistics
`finance.py`	Logs expenses and tracks cash flow

1.2 Business Rules Under Test

- A crew member must be registered before a role can be used.
- Only crew members with the `driver` role may enter a race.
- A car must exist in inventory before it can be used in a race.
- Race results update the cash balance in inventory.
- Missions cannot start if the required role is unavailable.
- Missions cannot start if there is insufficient cash.
- Finance expenses are deducted directly from inventory cash.
- Leaderboard rankings reflect race results correctly.

2 Test Suite 1: Registration & Crew Management

Modules involved: `registration.py`, `crew_management.py`

Purpose: Validates that crew management correctly reads member data from registration, and that skill and role operations respect the registered state of members.

Scenario	Expected Result	Actual Result	Status
Assign skill to a registered member	"Mohamad's skill level is now 80"	"Mohamad's skill level is now 80"	**PASS**
Assign skill to an unregistered member	"Member isn't registered"	"Member isn't registered"	**PASS**
Assign skill out of valid range (150)	"Skill level must be between 0 and 100(inclusive)"	"Skill level must be between 0 and 100(inclusive)"	**PASS**
Get role of registered member	"driver" (lowercase)	"driver"	**PASS**
Get role of unregistered member	`None`	`None`	**PASS**
Get skill level of registered member after update	80	80	**PASS**
Get skill level of unregistered member	"Member isn't registered"	"Member isn't registered"	**PASS**
Get members filtered by role (driver)	["Mohamad", "Sara"]	["Mohamad", "Sara"]	**PASS**
Get members by role with no matches	[]	[]	**PASS**

Errors Found

- **Bug (f-string missing):** `update_skill()` returned the literal string `{name}'s skill level is now {skill_level}` instead of formatting the values. Fixed by adding the `f` prefix to the return string.
- **Bug (typo):** `member_exits` was used throughout `crew_management.py` instead of `member_exists`. Fixed by correcting the function name in both the definition and all call sites.
- **Bug (wrong return type):** `get_role()` returned the string `"Member isn't registered"` for unregistered members instead of `None`, which broke the driver check in `race.py`. Fixed by returning `None`.

- **Bug (case normalisation):** Roles were stored as-is (e.g. `"Driver"`) but comparisons used lowercase `"driver"`. Fixed by calling `role.lower()` at registration time.

3 Test Suite 2: Race, Crew Management & Inventory

Modules involved: `race.py`, `crew_management.py`, `inventory.py`

Purpose: Validates that race creation correctly enforces the driver-only rule (via `crew_management`) and the car availability rule (via `inventory`).

Scenario	Expected Result	Actual Result	Status
Create race with valid driver and car	"Race:racetoglory has been created"	"Race:racetoglory has been created"	**PASS**
Create race with no car in inventory	"Ferari doesn't exist in inventory..."	"Ferari doesn't exist in inventory..."	**PASS**
Create race with unregistered driver	"Ahemad isn't a driver..."	"Ahemad isn't a driver..."	**PASS**
Create race with neither driver nor car	"Ahemad isn't a driver..."	"Ahemad isn't a driver..."	**PASS**
Create a race with a duplicate name	"Race already exists"	"Race already exists"	**PASS**

Errors Found

- **Bug (list vs dict indexing):** Inside the `for race in races` loop, the code used `races["name"]` (indexing the list) instead of `race["name"]` (indexing the current dict element). This caused a `TypeError` at runtime. Fixed in both `create_race()` and `get_race()`.
- **Bug (reset fixture):** The test fixture used `inventory.clear()` which removed all keys including `"cars"`, causing a `KeyError`. Fixed by resetting each key individually.
- **Bug (wrong argument type):** One test accidentally passed lists instead of strings to `create_race()`, causing a `TypeError: unhashable type: list`. Fixed by passing plain strings.

4 Test Suite 3: Results, Race & Inventory

Modules involved: `results.py`, `race.py`, `inventory.py`

Purpose: Validates that recording race results correctly reads from `race.py` and updates the cash balance in `inventory.py`. Also validates input checks for position and prize money.

Scenario	Expected Result	Actual Result	Status
Record race result with valid inputs	"Drive:Ahemad has finished..."	"Drive:Ahemad has finished..."	**PASS**
Record result for non-existent race	"Race doesn't exist"	"Race doesn't exist"	**PASS**
Record result for race that hasn't started	"Race isn't ongoing..."	"Race isn't ongoing..."	**PASS**
Record result with invalid position (-1)	"Invalid position..."	"Invalid position..."	**PASS**
Record result with negative prize money	"Invalid prize money..."	"Invalid prize money..."	**PASS**
Verify cash updates after recording result	1800 (1000 base + 800 prize)	1800	**PASS**

Errors Found

- **Bug (`append` syntax):** `results.append[...]` used square brackets instead of parentheses, causing a `TypeError` every time a result was recorded. Fixed to `results.append(...)`.
- **Missing validation:** `results.py` had no check for negative positions or negative prize money. Added guards at the top of `record_race()` to return appropriate error messages.

5 Test Suite 4: Missions, Crew Management & Inventory

Modules involved: `missions.py`, `crew_management.py`, `inventory.py`

Purpose: Validates that mission assignment checks role availability via `crew_management` and cash availability via `inventory`, and that the cost is correctly deducted.

Scenario	Expected Result	Actual Result	Status
Assign mission with valid inputs	"Mission:Bombthem has been assigned to Ahemad"	"Mission:Bombthem has been assigned to Ahemad"	**PASS**
Assign mission with insufficient cash	"Mission failed: Not enough cash"	"Mission failed: Not enough cash"	**PASS**
Assign mission with no available role	"Mission failed: No mechanic available"	"Mission failed: No mechanic available"	**PASS**
Assign mission, verify first available member is assigned	"assigned to Ahemad"	"assigned to Ahemad"	**PASS**
Assign mission and verify cash is deducted	933 (1000 − 67)	933	**PASS**

Errors Found

No errors found in this module interaction. All business rules were enforced correctly across the three modules.

6 Test Suite 5: Leaderboard & Results

Modules involved: `leaderboard.py`, `results.py`

Purpose: Validates that leaderboard correctly reads rankings and race history from `results.py`, and that sorting and statistics are accurate.

Scenario	Expected Result	Actual Result	Status
Show leaderboard after one race	[('Ahemad', 9)]	[('Ahemad', 9)]	**PASS**
Show leaderboard with no races	"No driver is ranked yet"	"No driver is ranked yet"	**PASS**
Show leaderboard with two drivers, same points	Both listed with 9 points	Both listed with 9 points	**PASS**
Show leaderboard sorted by points	Ahemad (9) then Hamza (8)	Ahemad (9) then Hamza (8)	**PASS**
Get top driver after two races	"Ahemad"	"Ahemad"	**PASS**
Get top driver with no races	"No drivers is ranked yet"	"No drivers is ranked yet"	**PASS**
Get driver stats after one race	{races: 1, total_prize: 800}	{races: 1, total_prize: 800}	**PASS**
Get driver stats with no races	{races: 0, total_prize: 0}	{races: 0, total_prize: 0}	**PASS**
Get driver stats after two races	{races: 2, total_prize: 1300}	{races: 2, total_prize: 1300}	**PASS**

Errors Found

- **Bug (function name mismatch):** `leaderboard.py` imported `get_rankings` but `results.py` defined the function as `get_ranking` (without the `s`). Fixed by renaming to `get_rankings` in `results.py`.
- **Bug (wrong assertion):** The initial test checked `assert ["Ahemad"] == result` but `show_leaderboard()` returns a list of tuples `(name, points)`. Fixed the assertion to `assert result == [("Ahemad", 9)]`.

7 Test Suite 6: Finance & Inventory

Modules involved: `finance.py`, `inventory.py`

Purpose: Validates that logging expenses correctly deducts from inventory cash via `inventory.py`, and that the summary accurately reflects remaining cash and total spent.

Scenario	Expected Result	Actual Result	Status
Log valid expense	"Expense logged:Car Repair, costed:200"	"Expense logged:Car Repair, costed:200"	**PASS**
Log expense with negative amount	"Amount can't be negative"	"Amount can't be negative"	**PASS**
Log expense exceeding available cash	"Expense failed: Not enough cash"	"Expense failed: Not enough cash"	**PASS**
Get summary after two expenses	cash: 700, spent: 300, count: 2	cash: 700, spent: 300, count: 2	**PASS**

Errors Found

- **Bug (expense not deducting cash):** The original `log_expense()` only appended to the expense list but never called `deduct_cash()`. This meant the inventory cash never changed. Fixed by calling `deduct_cash(amount)` before appending the expense entry.

8 Summary of All Errors Found

#	File	Error	Fix
1	registration.py	Typo: `member_exits` instead of `member_exists`	Renamed function and updated all call sites
2	crew_management.py	Missing `f` prefix on return string in `update_skill()`	Added `f` prefix to format the string
3	crew_management.py	`get_role()` returns string instead of `None` for unregistered members	Changed to return `None`
4	registration.py	Role stored as-is, causing case mismatch in comparisons	Store `role.lower()` at registration time
5	race.py	`races["name"]` should be `race["name"]` inside for loop	Fixed in `create_race()` and `get_race()`
6	results.py	`results.append[...]` used square brackets instead of parentheses	Changed to `results.append(...)`
7	results.py	Function named `get_ranking` but imported as `get_rankings`	Renamed to `get_rankings`
8	results.py	No validation for negative position or prize money	Added guards returning appropriate error messages
9	finance.py	`log_expense()` did not call `deduct_cash()`	Added `deduct_cash(amount)` call before appending
10	test files	`inventory.clear()` wiped all keys causing `KeyError`	Reset each key individually in fixture

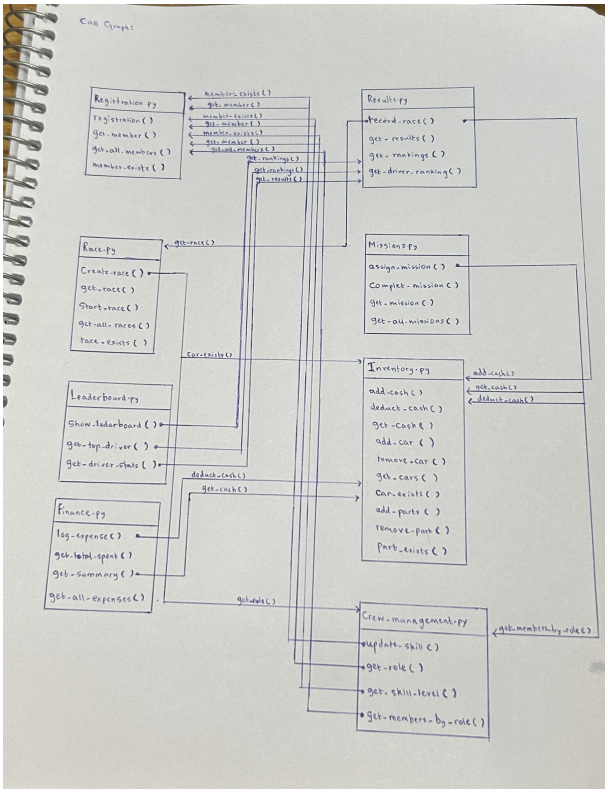

Figure 1: Enter Caption

9 Test Results Summary

Test Suite	Tests	Result
Registration & Crew Management	9	**PASS**
Race, Crew Management & Inventory	5	**PASS**
Results, Race & Inventory	6	**PASS**
Missions, Crew Management & Inventory	5	**PASS**
Leaderboard & Results	9	**PASS**
Finance & Inventory	4	**PASS**
Total	**38**	**38 passed, 0 failed**

All 38 integration tests pass after the errors listed in Section 7 were identified and corrected. Each test validates a cross-module interaction rather than isolated unit behaviour, confirming that data flows correctly between modules and all business rules are enforced at the boundaries.